JAIN DEEMED-TO-BE UNIVERSITY | ONLINE

E-Learning Material

Computer Architecture

(25VBT0C104)

www.onlinejain.com

Module: 05

Performance, Processor Technologies and Storage

Module Information

Field	Details
Course Title	Computer Architecture
Course Code	25VBT0C104
Module Number	Module 5
Module Title	Performance, Processor Technologies and Storage
BTL Level	BTL 4 (Analyse), BTL 5 (Evaluate) & BTL 6 (Create)
Semester	I
Credits	4

Module Overview

Module 5 is the capstone of this Computer Architecture course, bringing together all prior knowledge to evaluate, compare, and reason about complete, real-world computing systems. The module operates at the highest levels of Bloom's taxonomy — Analyse, Evaluate, and Create — challenging you to move beyond recall and comprehension toward critical assessment and design thinking.

The module opens with performance measurement — the quantitative foundation that allows architects to compare designs objectively. You will work with metrics such as MIPS, MFLOPS, CPI, and benchmark suites, and apply Amdahl's Law and Gustafson's Law to evaluate the limits and potential of performance improvement strategies including parallel execution, program partitioning, and task scheduling.

A significant section examines the major processor technology families that have shaped computing history: CISC and RISC architectures are revisited at a deeper level, followed by superscalar and VLIW processors that exploit instruction-level parallelism without (or with compiler help), and vector and symbolic processors that dominate scientific computing and artificial intelligence workloads.

The module then consolidates memory system knowledge by examining the complete memory hierarchy — from on-chip SRAM caches through DRAM main memory to secondary storage — with special focus on advanced topics: cache addressing modes, associative cache designs, TLB management, virtual-to-physical address translation through paging and segmentation, and SIMD memory access patterns.

The final section addresses storage systems and I/O infrastructure — the components that persist data and connect the CPU to the external world. RAID technology is examined in depth: all standard levels (0 through 6) are evaluated for their performance, capacity, and reliability trade-offs. I/O interfacing standards (PCI Express, SATA, NVMe, USB) and modern storage technologies (SSD, NVMe, cloud storage) complete the module.

By completing this module, you will be able to evaluate processor architectures, design memory systems, and assess storage configurations for real computing applications — skills that are directly applicable to careers in systems design, cloud infrastructure, and embedded systems engineering.

Learning Objectives

By the end of this module, students will be able to:

- Evaluate processor performance using quantitative metrics (MIPS, CPI, MFLOPS) and apply Amdahl's and Gustafson's Laws to analyse speedup limits.
- Analyse and compare CISC, RISC, superscalar, VLIW, vector, and symbolic processor architectures for specific application domains.
- Evaluate advanced memory system designs including cache addressing modes, associative caches, paging, segmentation, and TLB operation.
- Analyse SIMD processing architectures and evaluate their effectiveness for data-parallel workloads.
- Evaluate RAID levels and I/O interfacing technologies to design storage configurations appropriate for given performance and reliability requirements.

Learning Outcomes

Upon successful completion of this module, students will be able to:

- Calculate and compare MIPS, CPI, and MFLOPS values for given processor benchmarks, and apply Amdahl's/Gustafson's Law to evaluate real speedup scenarios.
- Compare CISC and RISC architectures and evaluate the design trade-offs of superscalar, VLIW, and vector processors for specific workloads.
- Differentiate between direct-mapped, set-associative, and fully associative cache addressing, and trace the virtual-to-physical address translation process through paging and segmentation.
- Evaluate SIMD processing advantages for data-parallel tasks and identify application domains where SIMD acceleration is most effective.
- Evaluate RAID 0 through RAID 6 configurations and recommend appropriate RAID levels for given reliability and performance requirements.

Table of Topics

Sub-Unit	Topic	Coverage
5.1	Performance Metrics and Speedup Laws	MIPS, MFLOPS, CPI, benchmarks, Amdahl's Law, Gustafson's Law
5.2	Program Partitioning and Scheduling	Task graphs, granularity, load balancing, scheduling strategies
5.3	Interconnection Networks	Bus, crossbar, mesh, hypercube, fat tree, dragonfly topologies
5.4	CISC and RISC Architectures	Deep comparison, microcode, compiler interaction, real examples
5.5	Superscalar and VLIW Processors	ILP, issue width, out-of-order, VLIW slots, Itanium
5.6	Vector and Symbolic Processors	Vector registers, stride, gather/scatter, LISP machines, AI hardware
5.7	Memory Hierarchy and Virtual Memory Technology	TLB, paging, segmentation, page fault handling, inverted page tables
5.8	Cache Addressing Modes and Associative Caches	Physical/virtual indexing, way prediction, VIPT, PIPT, VIVT
5.9	SIMD Processing	SSE, AVX, ARM NEON, GPU SIMT, data-parallel programming
5.10	Storage Systems and I/O Interfacing	HDD, SSD, NVMe, RAID 0–6, PCI Express, SATA, USB

5.1 Performance Metrics and Speedup Laws

Measuring computer performance accurately is critical for making informed design and purchasing decisions. A single metric rarely tells the complete story — architects use a suite of complementary measurements to evaluate systems from different angles.

5.1.1 Fundamental Performance Metrics

Clock Rate and Clock Cycle Time

The clock rate (in GHz) describes how many cycles occur per second. Clock cycle time (T_clk) is the reciprocal: T_clk = 1 / clock_rate. However, clock rate alone is a poor measure of performance — different processors accomplish different amounts of work per clock cycle.

CPI — Cycles Per Instruction

CPI measures the average number of clock cycles required to complete one instruction:

CPI = Total CPU Cycles / Total Instructions Executed

A lower CPI means more work per cycle. Modern superscalar processors can achieve CPI < 1 (multiple instructions per cycle). Stalls, cache misses, and branch mispredictions increase CPI.

CPU Execution Time

CPU Time = Instruction Count × CPI × T_clk

This "iron triangle" of performance shows three levers: reduce the number of instructions (better compiler/ISA), reduce CPI (better microarchitecture), or reduce clock cycle time (better fabrication process). Improving one may worsen another.

MIPS — Millions of Instructions Per Second

MIPS = Clock Rate / (CPI × 10^6)

- Limitation: MIPS is misleading when comparing different architectures. A RISC processor with more but simpler instructions may have higher MIPS but perform the same or less useful work than a CISC processor with fewer but more powerful instructions.
- Example: Processor A — 2 GHz, CPI = 2, MIPS = 1000. Processor B — 1 GHz, CPI = 0.5, MIPS = 2000. Processor B has higher MIPS but only half the clock rate.

MFLOPS — Millions of Floating-Point Operations Per Second

Used for scientific and numerical computing workloads. Modern supercomputers and GPUs are measured in TeraFLOPS (10^{12} FLOPS) or PetaFLOPS (10^{15} FLOPS).

Benchmarks

Benchmarks are standardised programs used to compare systems fairly:

- SPEC CPU2017: Industry-standard benchmark suite for integer and floating-point workloads. Used by Intel, AMD, and ARM to report processor performance publicly.
- LINPACK: Solves large dense linear systems; used to rank supercomputers on the TOP500 list.
- SPECjvm, SPECweb: Java and web server benchmarks.
- MLPerf: Benchmarks for machine learning training and inference — increasingly important as AI workloads dominate data centres.

5.1.2 Amdahl's Law

Amdahl's Law (Gene Amdahl, 1967) quantifies the maximum speedup achievable when only a fraction of a task can be parallelised:

$$\textbf{Speedup} = 1 / (s + (1 - s) / p)$$

Where s = sequential fraction (cannot be parallelised), p = number of processors, (1−s) = parallel fraction.

As p → ∞: Maximum Speedup → 1/s. This is the hard ceiling.

- Example: Program with 10% sequential code (s = 0.10). Max speedup = 1/0.10 = 10×, even with millions of processors.
- Implication: Reducing the sequential fraction is far more impactful than adding more processors. A program with s = 0.01 achieves max 100× speedup; s = 0.05 limits you to 20×.

5.1.3 Gustafson's Law — Scaled Speedup

Gustafson's Law (1988) challenges Amdahl's pessimism by observing that with more processors, programmers typically solve larger problems (not the same problem faster). When the problem size scales with processor count:

$$\textbf{Scaled Speedup} = p - s \times (p - 1) = s + p \times (1 - s)$$

- Example: 1000 processors, 1% sequential fraction. Scaled Speedup = 1000 − 0.01 × (999) ≈ 990×.
- Practical relevance: Weather simulation, drug discovery, and fluid dynamics scale problem size with hardware — Gustafson's model is more representative of these real HPC workloads.

Did You Know?

India's PARAM Yuva II supercomputer (C-DAC, Pune, 2013) achieved 524 TeraFLOPS peak performance. Its successor, PARAM Siddhi-AI (2020), delivers 5.267 PetaFLOPS (AI) using 210 NVIDIA A100 GPUs. PARAM Rudra (2024), installed at IIT Roorkee, IIT Kharagpur, and IISER Pune, marks the next generation of India's National Supercomputing Mission (NSM), targeting ExaFLOP-class systems by 2027.

✓ Let's Check

Q.No	Type	Question
1	MCQ	A processor runs at 3 GHz with CPI = 1.5. What is its MIPS rating? (A) 500 MIPS (B) 1000 MIPS (C) 2000 MIPS (D) 4500 MIPS
2	MCQ	If 25% of a program is sequential, what is the maximum speedup according to Amdahl's Law? (A) 25× (B) 10× (C) 4× (D) 8×
3	Fill in Blank	CPU Execution Time = Instruction Count × CPI × _____.
4	Fill in Blank	Gustafson's Law models the case where the _____ scales proportionally with the number of processors.
5	True/False	A processor with higher MIPS always performs better than one with lower MIPS. (True/False)
6	True/False	Amdahl's Law implies that reducing the sequential fraction has more impact on speedup than adding more processors. (True/False)

Answers:

Q.No	Answer
1	(C) 2000 MIPS — MIPS = Clock Rate / (CPI × 10^6) = 3×10^9 / (1.5 × 10^6) = 2000 MIPS.
2	(C) 4× — Max Speedup = 1/s = 1/0.25 = 4×.
3	T_clk (Clock Cycle Time)

4	Problem size (workload)
5	False — MIPS is architecture-dependent. Higher MIPS on a RISC processor may not mean more useful work than a CISC processor with lower MIPS.
6	True — Halving the sequential fraction doubles the maximum achievable speedup.

5.2 Program Partitioning and Scheduling

For a program to benefit from multiple processors, it must be divided into tasks that can execute concurrently. This division is called program partitioning. Once partitioned, a scheduler assigns tasks to processors to maximise utilisation and minimise total execution time.

5.2.1 Task Graphs

A task graph (or task dependency graph) is a directed acyclic graph (DAG) where:

- Nodes represent tasks (units of computation).
- Directed edges represent data dependencies — an edge from task A to task B means B cannot start until A completes.
- Node weights represent task execution time.
- Edge weights represent communication time (data transfer between tasks on different processors).

The critical path of a task graph is the longest weighted path from start to finish. The critical path length is the minimum possible execution time, regardless of how many processors are available.

5.2.2 Granularity

Granularity describes the ratio of computation time to communication time in a parallel program:

- Fine-grained parallelism: Many small tasks with frequent inter-task communication. High overhead; effective only with low-latency interconnects (shared memory multiprocessors).
- Coarse-grained parallelism: Few large tasks with infrequent communication. Lower overhead; better suited for distributed memory systems where communication is costly.
- Medium-grained: The practical middle ground used in most MPI applications.

5.2.3 Load Balancing

Load balancing ensures that all processors are busy for as much of the execution time as possible. An imbalanced distribution — where some processors finish early and sit idle while others are still working — wastes parallel resources and reduces speedup.

- Static Load Balancing: Tasks are assigned to processors before execution begins, based on estimated workloads. Simple but inflexible — if estimation is wrong, imbalance occurs.
- Dynamic Load Balancing: Tasks are assigned (or reassigned) at runtime based on actual processor loads. More adaptive but adds runtime overhead. Used in work-stealing schedulers (Java ForkJoin, Cilk, OpenMP).

5.2.4 Scheduling Strategies

- List Scheduling: Maintain a priority queue of ready tasks (dependencies satisfied). Schedule the highest-priority ready task to the next available processor. Widely used for static task scheduling.
- Cluster Scheduling: Tasks are grouped into clusters that execute on the same processor to minimise communication overhead. Effective for tasks with high mutual communication.
- Work Stealing: Idle processors "steal" tasks from the queues of busy processors. Used in Java's ForkJoinPool and Intel TBB (Threading Building Blocks). Excellent for recursive divide-and-conquer algorithms.

Did You Know?

Infosys's AI platform (Infosys Topaz) and TCS's AI Cloud platform use dynamic work-stealing schedulers to distribute deep learning inference tasks across thousands of GPU cores in their data centres across Bengaluru, Hyderabad, and Pune. This approach typically achieves 85–92% GPU utilisation — far above the industry average of 60–65%.

5.3 Interconnection Networks

The interconnection network connects processors to each other and to memory in a parallel computer. Its topology (physical layout), bandwidth, and latency directly determine the scalability and communication performance of the system. Choosing the right network topology is as critical a design decision as choosing the processor.

5.3.1 Key Network Metrics

- Bisection Bandwidth: The minimum bandwidth across any partition that divides the network into two equal halves. A high bisection bandwidth ensures that all-to-all communication remains fast.
- Diameter: The maximum number of hops between any two nodes. Smaller diameter = lower worst-case communication latency.
- Node Degree: The number of links connected to each node (router). Higher degree = more connectivity but more expensive hardware.

5.3.2 Common Topologies

Topology	Diameter	Bisection BW	Used In
Bus	1	Very Low	Small workstations, PCIe bus
Crossbar Switch	1	Very High	High-end switches, GPU NVLink
2D Mesh	$2(\sqrt{N}-1)$	Medium	Intel Xeon Phi, early Tilera chips
Torus	$\sqrt{N}$	Medium-High	Cray XT/XE series, Blue Gene
Hypercube	$\log_2 N$	High	CM-2, older MPP systems
Fat Tree	2 log N	High	InfiniBand clusters, Amazon AWS
Dragonfly	3	Very High	Cray XC series, Fugaku supercomputer

5.3.3 Fat Tree — The Industry Standard

The fat tree topology uses multiple layers of switches, with progressively "fatter" (higher-bandwidth) links toward the top of the tree. This ensures that communication between any two nodes passes through equally wide links, eliminating bottlenecks. Fat tree networks (implemented via InfiniBand HDR/NDR) are the dominant choice for HPC clusters and cloud data centres.

5.3.4 Dragonfly

The dragonfly topology arranges routers into groups (cliques), with all-to-all connectivity within each group and sparse global connectivity between groups. This

achieves very low diameter (typically 3 hops between any two nodes) with moderate cabling cost — the reason it is used in the world's largest supercomputers.

🔍 Analysing Misconceptions: More Links = Better Network Performance

❌ MYTH: Adding more physical links between nodes always improves network performance.

✅ REALITY: Beyond a certain point, adding links increases cost and complexity without proportional performance gain. The bisection bandwidth is the actual bottleneck for many parallel workloads. Additionally, routing algorithm complexity grows with link count, potentially increasing latency for small messages. Modern supercomputers use carefully engineered topologies (dragonfly, slim fly) that optimise the bandwidth-per-dollar metric rather than raw link count.

5.4 CISC and RISC Architectures

The CISC (Complex Instruction Set Computer) vs RISC (Reduced Instruction Set Computer) debate is one of the most consequential in computer architecture history. Understanding both philosophies — and their modern convergence — is essential for evaluating processor designs.

5.4.1 CISC Philosophy

CISC architectures (Intel x86, Motorola 68000) were designed in the 1960s–1970s when compilers were primitive and programmers wrote assembly by hand. The goal was to provide powerful, memory-efficient instructions that could accomplish complex operations in a single instruction.

- Variable-length instructions (1–15 bytes in x86) allow compact code.
- Many addressing modes (up to 25 in some architectures) give programmers flexibility.

- Memory-to-memory operations: instructions can directly read from and write to memory.
- The control unit is typically microprogrammed to handle the complexity of decoding and executing variable-length instructions.
- Key example: x86 REP MOVS instruction can move an entire block of memory in one assembly statement — a task requiring a loop in RISC.

5.4.2 RISC Philosophy

RISC (Patterson and Hennessy, UC Berkeley and Stanford, ~1980) was driven by empirical studies showing that compilers used only a small subset of CISC instructions, and that simpler instructions executed faster due to pipeline efficiency.

- Fixed-length instructions (32-bit): IF stage always fetches exactly one instruction per cycle.
- Load/store architecture: only LOAD and STORE access memory; all computation on registers.
- Large register file (typically 32 registers): reduces memory accesses.
- Simple addressing modes: 2–3 modes only; hardware complexity shifts to the compiler.
- Hardwired control: clean, fast pipeline without microcode overhead.

5.4.3 Modern Convergence

The distinction has blurred significantly in modern processors:

- Modern x86 processors (Intel Core Ultra, AMD Ryzen) internally translate complex x86 instructions into RISC-like micro-operations (μops) before pipelining. The front-end is CISC; the back-end execution engine is RISC.
- ARM processors (RISC) added complex instructions (Thumb-2, NEON, SVE) to handle multimedia and signal processing efficiently — moving toward CISC complexity.

- The net result: at the silicon level, virtually all modern high-performance processors use a RISC-style execution core, regardless of their ISA.

Feature	CISC	RISC
Instruction Length	Variable (1–15 bytes)	Fixed (32 bits)
Memory Access	Any instruction	LOAD/STORE only
Control Unit	Microprogrammed	Hardwired
Registers	8–16 (x86 historically)	32–64 general purpose
Pipeline Complexity	High (variable decode)	Low (clean stages)
Examples	Intel x86, Motorola 68000	ARM, RISC-V, MIPS, PowerPC

Did You Know?

The ARM processor ISA — RISC-based — now powers over 95% of mobile devices worldwide. In India, virtually every smartphone (from budget Redmi A3 with MediaTek Helio to flagship Samsung Galaxy S24 with Snapdragon 8 Gen 3) runs an ARM core. India's smartphone production (over 330 million units in FY2023–24, per ICEA data) means ARM architecture is the most deployed processor design in the country by a very wide margin.

5.5 Superscalar and VLIW Processors

Both superscalar and VLIW processors exploit instruction-level parallelism (ILP) — the ability to execute multiple instructions simultaneously. They differ fundamentally in who is responsible for finding and managing this parallelism: the hardware (superscalar) or the compiler (VLIW).

5.5.1 Superscalar Processors

A superscalar processor issues multiple instructions per clock cycle by having multiple execution units (multiple ALUs, floating-point units, load/store units) running in parallel. The hardware dynamically detects independent instructions and dispatches them to available execution units.

Key Features

- Issue Width: The number of instructions that can be issued per cycle. Modern processors issue 4–8 instructions per cycle (4-way to 8-way superscalar).
- Out-of-Order Execution (OOO): Instructions are not executed in program order. The processor dynamically reorders them to avoid stalls, using a Reorder Buffer (ROB) to track in-flight instructions and commit results in order.
- Register Renaming: Physical registers are mapped to logical registers dynamically. This eliminates WAR and WAW hazards by giving each instruction write its own physical destination register.
- Tomasulo's Algorithm: The classic algorithm (IBM 360/91, 1967) for dynamic instruction scheduling using reservation stations and a Common Data Bus (CDB). It is the theoretical foundation of all modern out-of-order processors.

Performance Example

Intel Core i9-14900K (2023): 8 Performance-cores + 16 Efficiency-cores. The P-cores have a 6-wide superscalar out-of-order execution engine with 512-entry ROB, capable of retiring up to 6 µops/cycle — among the widest in any commercial processor.

Limitations

- Hardware complexity: Dynamic scheduling, ROB, and register renaming require significant transistors and power.
- Diminishing returns: Finding truly independent instructions becomes increasingly difficult as issue width grows. Memory dependencies and control flow limit exploitable ILP.

5.5.2 VLIW — Very Long Instruction Word

VLIW takes the opposite approach: instead of hardware dynamically finding parallelism, the compiler explicitly packages multiple independent operations into a single very long instruction word. Each slot in the VLIW word targets a specific functional unit.

- Compiler responsibility: The compiler analyses the program, identifies independent operations, and packs them into VLIW words at compile time.
- Simple hardware: No dynamic scheduling, ROB, or complex dispatch logic needed — hardware is simpler and more power-efficient.
- VLIW word format: A 256-bit VLIW word might contain: [ALU1 op] [ALU2 op] [FPU op] [Load/Store op] [Branch op] — all executing simultaneously.
- NOP problem: When the compiler cannot find enough independent operations to fill all slots, it must insert NOPs (no-operation), wasting issue slots and code space.

Feature	Superscalar	VLIW
ILP Detection	Hardware (dynamic)	Compiler (static)
Scheduling	Runtime, out-of-order	Compile time, in-order
Hardware Complexity	Very High	Low
Binary Compatibility	Good — same binary on new chips	Poor — recompile for new hardware
Examples	Intel Core, AMD Ryzen, ARM Cortex-A	Intel Itanium (IA-64), TI C6000 DSP

🔍 Analysing Misconceptions: Superscalar Processors and IPC

✗ MYTH: A 4-way superscalar processor always executes exactly 4 instructions per cycle.

✓ REALITY: The 4-way designation means the processor can issue up to 4 instructions per cycle — it represents peak throughput. In practice, due to data hazards, cache misses, branch mispredictions, and resource conflicts, the actual Instructions Per Cycle (IPC) is typically 1–3 for a 4-way superscalar. Real-world IPC heavily depends on the instruction mix and memory access patterns of the workload.

5.6 Vector and Symbolic Processors

Beyond the general-purpose scalar and superscalar processors lies a world of specialised architectures designed to excel at specific classes of computation. Vector processors dominate scientific computing; symbolic processors were developed for artificial intelligence and knowledge representation. Understanding both broadens the picture of what is possible when hardware is matched to its workload.

5.6.1 Vector Processors

A vector processor operates on entire arrays (vectors) of data with a single instruction, rather than processing one element at a time. This is the architectural embodiment of data-level parallelism for scientific workloads such as weather modelling, computational fluid dynamics, and numerical linear algebra.

Key Concepts

- Vector Register: A special register that holds multiple data elements (e.g., 64 double-precision floating-point values). The Cray-1 (1976) had 8 vector registers, each 64 elements × 64 bits.
- Vector Instruction: VADD V1, V2, V3 adds all corresponding elements of vectors V2 and V3, storing results in V1 — in parallel. One instruction replaces a loop of scalar additions.
- Stride: Vectors are not always stored in contiguous memory. Stride defines the spacing between elements: stride-1 = contiguous; stride-n = every nth memory location. Hardware supports non-unit strides directly.
- Gather/Scatter: Load (gather) or store (scatter) vector elements from/to non-contiguous memory locations specified by an index vector. Critical for sparse matrix operations.
- Chaining: Multiple vector operations can be pipelined ("chained") so that the output of one feeds directly into the next without storing intermediate results to memory.

Modern Vector Extensions

Vector processing principles are embedded in modern processors through SIMD extensions:

- Intel AVX-512: 512-bit vector registers; operate on 16 × 32-bit floats or 8 × 64-bit doubles simultaneously. Used in Intel Xeon Scalable processors for HPC and AI workloads.
- ARM SVE (Scalable Vector Extension): Variable-length vector registers (128–2048 bits). Used in ARM Neoverse V1/V2 (powering AWS Graviton3, Alibaba Cloud Yitian 710).
- RISC-V Vector Extension (RVV): Standard vector extension for RISC-V, supporting variable-length vectors. The basis of SiFive's X280 vector core and C-DAC's Vega processor family.

5.6.2 Symbolic Processors

Symbolic processors (also called AI processors or LISP machines) were designed in the 1970s–1980s to efficiently execute symbolic computation — programs that manipulate symbols, lists, and logical rules rather than numbers. They were tailored for AI research languages such as LISP, Prolog, and Smalltalk.

Key Features

- Tagged Architecture: Every data word includes a type tag (integer, float, symbol, list pointer). Hardware checks tags on every operation, enabling dynamic type checking without software overhead.
- Garbage Collection Hardware: LISP programs create and discard many temporary data structures. Dedicated GC hardware identified unused memory and reclaimed it efficiently.
- List Processing Instructions: Native hardware support for CAR (head of list), CDR (tail of list), and CONS (construct new list pair) operations — the fundamental operations of LISP.

- Examples: Symbolics 3600 (1981), MIT CADR machine, Xerox Alto (partial symbolic support).

Legacy and Revival

While dedicated LISP machines faded with the rise of faster general-purpose workstations, their concepts live on in modern AI accelerators:

- Google TPU, AWS Trainium, and NVIDIA H100 Tensor Core all incorporate data-type tagging (bfloat16, TF32, FP8) and hardware support for tensor operations — the modern equivalent of symbolic type specialisation.
- India's C-DAC Vega AI processor (2022) incorporates vector and AI-specific execution units for inference workloads in healthcare diagnostics and smart city applications under the National AI Mission.

Did You Know?

Japan's Fugaku supercomputer — ranked #1 in the world on the TOP500 list in 2020–2021 — uses ARM A64FX processors with 512-bit SVE vector units. Each A64FX chip contains 48 compute cores and achieves over 3 TeraFLOPS per chip. Fugaku was used to simulate SARS-CoV-2 airborne droplet transmission in 2020, directly informing social distancing policy in Japan and providing data cited by WHO guidelines.

5.7 Memory Hierarchy and Virtual Memory Technology

Module 3 introduced the memory hierarchy and cache fundamentals. This module examines advanced virtual memory mechanisms — segmentation, multi-level paging, TLB management, and page replacement — at the depth needed to evaluate and design memory systems for real operating systems and processors.

5.7.1 Paging — Revisited at Depth

Recall that paging divides virtual and physical memory into fixed-size pages/frames. Modern operating systems extend this with multi-level page tables to handle very large virtual address spaces efficiently.

Multi-Level Page Tables

A 64-bit virtual address space would require a single-level page table with 2^{52} entries (for 4 KB pages) — roughly 32 petabytes just for the page table. Modern processors use multi-level page tables (4-level in x86-64, up to 5-level in recent Intel processors):

- x86-64 four-level paging: Virtual address → [PGD index | PUD index | PMD index | PT index | Page Offset].
- Each level points to a table in memory. Only tables for actually used virtual addresses are allocated — saving enormous amounts of memory.
- Linux uses four-level page tables (PGD → PUD → PMD → PT) for 64-bit processes. Five-level paging extends the addressable space to 128 PB.

TLB Management

- TLB Shootdown: In a multiprocessor system, when one processor modifies a page table entry, it must invalidate the corresponding TLB entries on ALL other processors that might have a cached copy. This requires an inter-processor interrupt (IPI) and is a significant overhead in multi-core systems.
- ASID (Address Space Identifier): A tag added to TLB entries identifying which process's address space they belong to. Allows TLB entries from different

processes to coexist — eliminating the need for a full TLB flush on every context switch. Used in ARM and RISC-V processors.

5.7.2 Segmentation

Segmentation divides the virtual address space into variable-sized logical segments — each corresponding to a program component such as the code segment, data segment, stack, and heap. A segment table maps segment numbers to base addresses and limits.

- Virtual address = [Segment Number | Offset]. The MMU checks: Offset < Segment Limit (prevents out-of-segment access → segmentation fault).
- Physical Address = Segment Base + Offset.

Segmentation vs Paging

Feature	Paging	Segmentation
Unit Size	Fixed (4 KB typical)	Variable (programmer-defined)
External Fragmentation	None	Yes — gaps between segments
Internal Fragmentation	Yes — last page may be partly empty	None
Protection	Page-level (coarse)	Segment-level (semantic boundary)
OS Usage	Universal (Linux, Windows, macOS)	Intel x86 legacy; Multics historical

5.7.3 Inverted Page Tables

Traditional page tables have one entry per virtual page — growing with the virtual address space size. An inverted page table has one entry per physical frame — growing only with physical memory size. This dramatically reduces page table memory for large virtual address spaces.

- Each entry stores: (Process ID, Virtual Page Number) → Physical Frame Number.
- Lookup requires searching the entire inverted table (or using a hash function for efficiency).
- Used in: IBM POWER architecture, early HP-UX.

Did You Know?

The Linux kernel's memory management subsystem handles page tables for billions of processes worldwide. On Indian cloud infrastructure (Jio Cloud, AWS Mumbai region, Azure India Central), a single physical server may host 200–500 virtual machines, each with its own four-level page table. Linux's Transparent Huge Pages (THP) feature merges adjacent 4 KB pages into 2 MB huge pages, reducing TLB pressure by 512× for memory-intensive workloads like Java application servers common in India's IT services sector.

✓ Let's Check

Q.No	Type	Question
1	MCQ	Which feature of a TLB eliminates the need for a full TLB flush on every process context switch? (A) TLB Shootdown (B) ASID tags (C) Dirty bits (D) Valid bits
2	MCQ	A segmentation fault occurs when a program accesses a virtual address whose offset exceeds: (A) The page size (B) The TLB capacity (C) The segment's defined limit (D) The physical memory size
3	Fill in Blank	An inverted page table has one entry per _____ rather than per virtual page.
4	Fill in Blank	In x86-64, _____ levels of page tables are used to handle the 64-bit virtual address space.
5	True/False	Paging eliminates external fragmentation but may suffer from internal fragmentation. (True/False)
6	True/False	A TLB shootdown is required when a page table entry is modified in a multiprocessor system. (True/False)

Answers:

Q.No	Answer
1	(B) ASID tags — Address Space Identifiers allow entries from different processes to coexist in the TLB without conflicts.
2	(C) The segment's defined limit — access beyond the limit triggers a segmentation fault exception.
3	Physical frame
4	Four (4)
5	True — unused space in the last page of a process is internal fragmentation.
6	True — all processors caching that TLB entry must be notified via an IPI to invalidate their stale entries.

5.8 Cache Addressing Modes and Associative Caches

While Module 3 covered cache mapping techniques (direct-mapped, set-associative, fully associative), this module examines a deeper question: should cache lookups use virtual addresses (faster, available before translation) or physical addresses (correct, available after translation)? This choice has profound implications for cache design, performance, and correctness.

5.8.1 Cache Addressing Schemes

PIPT — Physically Indexed, Physically Tagged

The cache is indexed and tagged using the physical address. The virtual address must first be translated by the MMU/TLB before the cache can be accessed.

- Advantage: Correct and unambiguous — no aliasing issues. Different virtual addresses that map to the same physical address always hit the same cache line.
- Disadvantage: TLB lookup must complete before cache lookup begins, adding TLB latency to the critical path.
- Used in: L2 and L3 caches in virtually all modern processors.

VIVT — Virtually Indexed, Virtually Tagged

Both the cache index and tag are derived from the virtual address. The cache can be accessed before TLB translation completes.

- Advantage: Fastest access — cache lookup starts immediately using the virtual address.
- Disadvantage: Aliasing — two virtual addresses from different processes may map to the same physical address but appear as different cache entries (homonym problem). Full cache flush required on context switch (extremely costly).
- Historically used in some SPARC and MIPS L1 caches; largely abandoned for correctness reasons.

VIPT — Virtually Indexed, Physically Tagged

The cache index uses virtual address bits, but the tag is the physical address. TLB lookup and cache indexing proceed in parallel — the TLB result (physical tag) arrives just in time to compare with the fetched cache tags.

- Advantage: Near-VIVT speed (parallel TLB + cache lookup) with PIPT correctness (physical tags prevent aliasing).
- Critical constraint: For VIPT to be alias-free, the index bits must not cross the page boundary. This means: cache size / associativity ≤ page size. For 4 KB pages and 4-way associativity: L1 cache must be ≤ 16 KB to guarantee alias-free VIPT.

- Used in: ARM Cortex-A7/A53/A78 L1 data caches (32 KB, 4-way = 8 KB per way ≤ 4 KB page if constrained; ARM uses a synonym check for safety), Intel Core L1 data cache (48 KB, 12-way).

5.8.2 Fully Associative Cache — Hardware Realities

Fully associative cache (any block can go anywhere) requires comparing the incoming address tag against ALL cache tags simultaneously. This is implemented using content-addressable memory (CAM):

- CAM: A memory array where each row stores a tag and has its own comparator. A lookup broadcasts the query tag to all rows simultaneously — all comparators fire in parallel, and a hit is detected in O(1) time regardless of cache size.
- Cost: CAM cells are 5–10× larger than SRAM cells. A 1024-entry fully associative cache using CAM would require enormous silicon area — impractical for large caches.
- Real use: Fully associative cache is only practical for very small, highly critical structures: TLBs (typically 32–1024 entries fully associative), victim caches (4–16 entries), and micro-TLBs.

5.8.3 Way Prediction

In a set-associative cache, a hit requires checking all k ways in a set. Way prediction uses a simple predictor (often just the last hit way) to guess which way is likely to contain the data. If correct, only one way is read — saving energy and reducing hit time. If wrong, the remaining ways are checked.

- Used in: ARM Cortex-A55, ARM Cortex-M7 caches to reduce dynamic power consumption by 30–50%.

🔍 Analysing Misconceptions: VIPT Cache and Correctness

❌ MYTH: VIPT caches always have aliasing issues and require a TLB flush on context switch.

✅ REALITY: VIPT caches with carefully constrained size (size/associativity ≤ page size) are completely alias-free and do not require TLB flushes on context switch. The ARM Cortex-A series uses 32 KB, 4-way L1 data caches with VIPT — operating correctly with 4 KB pages because 32 KB / 4 ways = 8 KB > 4 KB. ARM handles this through a synonym check during cache fill, maintaining correctness without full flushes.

5.9 SIMD Processing

SIMD (Single Instruction, Multiple Data) is the most widely deployed form of data parallelism in modern computing. From smartphone cameras to cloud AI inference, SIMD hardware executes the same operation on multiple data elements simultaneously, dramatically accelerating data-parallel workloads without the complexity of multi-core programming.

5.9.1 SIMD Fundamentals

A SIMD instruction operates on a vector register containing multiple data elements:

- Example: A 256-bit AVX register can hold 8 × 32-bit floats. The instruction VADDPS ymm0, ymm1, ymm2 adds all 8 float pairs simultaneously — 8 additions in one instruction.
- Without SIMD: 8 separate FADD instructions required, each consuming one instruction issue slot.
- With SIMD: 1 instruction, 8× throughput for floating-point additions.

5.9.2 SIMD Evolution

Extension	Year	Register Width	Max Parallelism
Intel MMX	1997	64 bits	8 × 8-bit integers
Intel SSE	1999	128 bits (XMM)	4 × 32-bit floats
Intel SSE2/3/4	2001–2007	128 bits	2 × 64-bit doubles / 16 × 8-bit ints
Intel AVX/AVX2	2011–2013	256 bits (YMM)	8 × 32-bit floats / 4 × 64-bit doubles
Intel AVX-512	2016–present	512 bits (ZMM)	16 × 32-bit floats / 8 × 64-bit doubles
ARM NEON	2005–present	128 bits	4 × 32-bit floats / 16 × 8-bit ints
ARM SVE/SVE2	2018–present	128–2048 bits (scalable)	Variable, scales with vector length

5.9.3 GPU SIMT — A Generalisation of SIMD

Modern GPUs use SIMT (Single Instruction, Multiple Threads) — a generalisation of SIMD where a single instruction is executed by many threads (a "warp" of 32 in NVIDIA GPUs), each operating on different data. Unlike SIMD, individual threads within a warp can take different code paths (though divergent threads reduce efficiency).

- NVIDIA A100 GPU: 6912 CUDA cores organised in 216 Streaming Multiprocessors, each executing 32-thread warps. Peak FP32 performance: 19.5 TeraFLOPS.
- Warp divergence: When threads in a warp take different branches (if-else), the warp executes both paths sequentially (with inactive threads masked off), halving throughput.

5.9.4 Application Domains

- Image and video processing: Each pixel is an independent data element; SIMD processes rows of pixels simultaneously.
- Audio processing: Dolby Atmos encoding in Indian multiplex chains (PVR, INOX) uses SIMD acceleration on Intel Core processors for real-time 128-channel audio mixing.
- Machine learning inference: Quantised neural networks (INT8, INT4) process 4–8× more operations per SIMD cycle than FP32 — the basis of MediaTek Dimensity 9300 APU performance in mid-range Indian smartphones.
- Cryptography: AES-NI (hardware AES), SHA-NI (hardware SHA) are SIMD-style extensions that accelerate encryption in VPNs, HTTPS, and UPI payment security (processed by NPCI on Intel Xeon servers).

Did You Know?

India's Unified Payments Interface (UPI), which processed over 13.9 billion transactions in a single month (March 2024, per NPCI data), relies heavily on SIMD-accelerated AES-256 encryption on NPCI's Intel Xeon Scalable server clusters in Mumbai and Hyderabad data centres. The AES-NI instruction set extension reduces encryption overhead by 10–15× compared to software AES, making real-time transaction processing at this scale feasible.

5.10 Storage Systems and I/O Interfacing

Storage systems are the persistent memory of a computer — they retain data even when power is removed. Understanding storage technologies, interfaces, and RAID configurations is essential for designing systems that are not only fast but also reliable. This is the final piece of the complete computer architecture picture.

5.10.1 Storage Technologies

Hard Disk Drive (HDD)

HDDs store data magnetically on spinning platters. A read/write head on a mechanical arm moves to the correct track and waits for the sector to rotate beneath it.

- Seek Time: Time to move the head to the correct track. Average: 3–10 ms.
- Rotational Latency: Time for the correct sector to rotate under the head. At 7200 RPM: average latency = 4.2 ms.
- Transfer Rate: Sequential transfer: 100–200 MB/s. Random IOPS: 100–200 operations/second.
- Use case: Bulk archival storage, video surveillance, NAS (Network Attached Storage). Cost: ₹2–4 per GB.

Solid State Drive (SSD) — SATA

SSDs use NAND flash memory — no moving parts. Access time is measured in microseconds rather than milliseconds.

- Random Read IOPS: 90,000–100,000 IOPS (vs 100–200 for HDD — 1000× faster for random access).
- Sequential Read: 500–550 MB/s (SATA III limit).
- Endurance: NAND cells wear out with write cycles. Enterprise SSDs specify endurance in DWPD (Drive Writes Per Day).

NVMe SSD (Non-Volatile Memory Express)

NVMe SSDs connect directly to the CPU via PCIe lanes, bypassing the SATA controller. This dramatically reduces latency and increases throughput.

- Protocol: NVMe replaces AHCI; reduces command queue depth overhead from 32 bytes (AHCI) to 8 bytes (NVMe) per command.
- Sequential Read: 3,500–7,000 MB/s (PCIe Gen 4); up to 14,000 MB/s (PCIe Gen 5).
- Latency: 20–100 µs (vs 100–1000 µs for SATA SSD).
- Used in: All modern gaming laptops, workstations, and cloud provider boot volumes (AWS gp3/io2, Azure Premium SSD v2).

5.10.2 I/O Interfaces

Interface	Peak Bandwidth	Primary Use
PCIe Gen 4 ×16	32 GB/s	GPU, NVMe RAID, high-end NICs
PCIe Gen 5 ×16	64 GB/s	AI accelerators, CXL memory expansion
SATA III	600 MB/s	SATA SSDs, optical drives
USB 3.2 Gen 2×2	2 GB/s	External SSDs, high-speed peripherals
Thunderbolt 4	5 GB/s	Docking stations, 8K display, external GPU
InfiniBand HDR	25 GB/s per port	HPC cluster interconnect

5.10.3 RAID — Redundant Array of Independent Disks

RAID technology combines multiple physical disks into a logical unit that improves performance, reliability, or both. Developed by Patterson, Gibson, and Katz at UC Berkeley in 1987, RAID has become the foundation of all enterprise storage systems.

RAID 0 — Striping (No Redundancy)

Data is striped across n disks in fixed-size blocks. A 2-disk RAID 0 splits every file alternately between disks, doubling sequential read/write throughput.

- Performance: Read/Write = n × single disk speed. IOPS ≈ n × single disk IOPS.
- Capacity: 100% utilisation — n × disk_size.
- Reliability: WORSE than a single disk. If any one disk fails, ALL data is lost. MTTF (Mean Time To Failure) = MTTF_single / n.
- Use case: Video editing scratch disks, gaming storage where speed matters and backups exist.

RAID 1 — Mirroring

Every write is duplicated to n disks (typically n=2). Each disk holds a complete copy of all data.

- Performance: Read throughput ≈ 2× (reads can be serviced by either disk). Write speed = single disk (must write to both).
- Capacity: 50% efficiency — only 1× disk_size usable despite 2× cost.
- Reliability: Survives failure of n−1 disks. MTTF improves dramatically.
- Use case: OS drives, database transaction logs, any data where loss is unacceptable.

RAID 5 — Striping with Distributed Parity

Data is striped across n disks with parity information distributed across all n disks (no dedicated parity disk). Parity = XOR of all data blocks in a stripe; it allows reconstruction of any single failed disk.

- Performance: Read = (n−1) × single disk. Write is slower due to parity calculation (read-modify-write penalty for random writes).
- Capacity: (n−1)/n × total capacity. For 5 disks: 80% efficiency.
- Reliability: Survives failure of any ONE disk. Data is rebuilt using parity during rebuild.
- Use case: Most popular enterprise RAID level — NAS (Synology, QNAP), ISCSI storage arrays.

RAID 6 — Double Parity

Like RAID 5 but with two independent parity calculations (typically P+Q using Reed-Solomon codes). Can survive simultaneous failure of any TWO disks.

- Performance: Slightly lower write performance than RAID 5 (two parity calculations).
- Capacity: (n−2)/n × total capacity. For 6 disks: 66.7% efficiency.
- Reliability: Tolerates simultaneous failure of any two disks — critical given that during RAID 5 rebuild (which can take 24–48 hours for large disks), a second disk failure destroys all data.
- Use case: High-capacity enterprise arrays, backup NAS systems with 4+ TB drives.

RAID 10 — Mirroring + Striping (RAID 1+0)

A combination of RAID 1 (mirroring) and RAID 0 (striping). First mirror pairs of disks, then stripe across the mirrors.

- Performance: Combines read throughput of RAID 0 with write speed close to RAID 1. Best performance among redundant RAID levels.
- Capacity: 50% efficiency (same as RAID 1).
- Reliability: Can survive multiple disk failures as long as no mirror pair loses both disks.
- Use case: High-performance databases (MySQL, Oracle), trading platforms, anywhere both speed and reliability are premium.

RAID	Disks Min	Capacity	Fault Tolerance	Best For
RAID 0	2	100% (n disks)	None	Speed-critical, non-critical data
RAID 1	2	50%	1 disk failure	OS, critical small datasets
RAID 5	3	(n−1)/n	1 disk failure	General enterprise NAS/SAN
RAID 6	4	(n−2)/n	2 simultaneous failures	Large arrays, archival storage
RAID 10	4	50%	Multiple (per mirror)	High-performance databases

🔍 Analysing Misconceptions: RAID as a Backup Solution

❌ MYTH: RAID protects your data — you do not need backups if you use RAID.

✅ REALITY: RAID provides hardware redundancy against disk failures but is NOT a backup. RAID does NOT protect against: accidental file deletion (deleted from all mirrors simultaneously), ransomware or malware (encrypts data on all RAID members), controller failure (can corrupt all disks), fire or flood (destroys the entire array). The 3-2-1 backup rule (3 copies, 2 media types, 1 offsite) is required alongside RAID for true data protection.

Case Study

📓 Case Study: Reliance Jio's JioBrain AI Infrastructure — Architecture Decisions Across the Stack

Background

Reliance Jio, India's largest telecom operator with over 490 million subscribers (as of Q4 FY2024), operates one of the world's largest AI inference infrastructures through its JioBrain platform. JioBrain powers real-time personalisation for JioTV, JioCinema, JioMart, and Jio's 5G network optimisation — processing over 100 billion events per day across Jio's data centres in Mumbai, Chennai, and Hyderabad.

Architecture Challenges Addressed

- Performance & SIMD: Real-time video recommendation for JioCinema (which streamed IPL 2023 to a record 32 million concurrent viewers) requires neural network inference on millions of user profiles per second. Jio's AI servers use AMD EPYC 9004 processors with AVX-512 SIMD, achieving 16× throughput improvement for INT8 matrix multiplication versus scalar code — directly enabling sub-10ms recommendation latency.
- Memory Hierarchy: JioBrain's recommendation model embeddings exceed 500 GB — far larger than any server's DRAM. Jio's team designed a three-tier memory strategy: hot embeddings (recently accessed users) in 1.5 TB DRAM (8 × 192 GB AMD 3DS RDIMM per server), warm embeddings in NVMe RAID (PCIe Gen 4 NVMe arrays, 7 GB/s sustained bandwidth), and cold embeddings on object storage. The cache hit rate for the DRAM tier exceeds 91%, keeping 91% of inference requests sub-millisecond.
- RAID and Storage: Video content for JioCinema (8 petabytes of encoded video as of 2023) is stored on Jio's custom object storage system running on 12-drive RAID 6 arrays (10+2 configuration, 72% capacity utilisation) using 18 TB Seagate Exos HDDs. This provides double-disk fault tolerance —

essential given that Jio operates over 50,000 storage drives across its three data centres. The expected mean time between data loss events exceeds 1 million years under the RAID 6 configuration.

- Interconnection & Parallel Processing: JioBrain's training cluster uses 256 NVIDIA H100 GPUs interconnected with InfiniBand NDR (400 Gbps per port) in a fat-tree topology. Amdahl's Law analysis showed the training pipeline was 12% sequential (checkpoint saving, data loading); achieving 8.3× maximum speedup with 256 GPUs — consistent with observed 7.9× real speedup (95% parallel efficiency).

Outcome

- JioBrain reduced content recommendation latency from 180 ms (2021, CPU-only) to under 8 ms (2024, AVX-512 + GPU), improving click-through rate on JioCinema by 23%.
- RAID 6 storage achieved 99.9999% data durability (six nines) across the 50,000-drive estate with zero data loss events in 2023.
- The fat-tree InfiniBand network achieved 94% bisection bandwidth utilisation during IPL 2023 peak traffic — a testament to good network topology selection.

Reference: Reliance Jio Annual Report 2023–24; RIL Technology Infrastructure Investor Presentation, March 2024; NASSCOM India Data Centre Report 2024.

Summary

Module 5 — the capstone of this course — has tied together all the threads of Computer Architecture into a comprehensive view of how modern computing systems are evaluated, designed, and deployed.

Performance Measurement:

- CPI, MIPS, MFLOPS, and benchmark suites provide quantitative tools for evaluating and comparing processor performance across workloads.
- Amdahl's Law sets hard limits on parallelism speedup based on the sequential fraction; Gustafson's Law provides a more optimistic view for scaled workloads. Reducing the sequential fraction is the most impactful performance lever.
- Program partitioning, task graphs, granularity, and dynamic work-stealing schedulers are the tools that translate theoretical parallelism into real speedup.

Processor Architectures:

- CISC and RISC represent different design philosophies; modern processors converge at the execution engine level, with RISC-style μop cores executing x86 macrocode.
- Superscalar processors dynamically find ILP using out-of-order execution, register renaming, and reservation stations. VLIW pushes this responsibility to the compiler.
- Vector processors exploit data-level parallelism for scientific workloads; symbolic processors handle AI and knowledge representation. Both principles live on in modern SIMD extensions and AI accelerators.

Memory Systems:

- Multi-level paging, TLBs with ASID tags, and inverted page tables are advanced mechanisms for managing large virtual address spaces efficiently in multi-core, multi-process environments.
- Segmentation provides semantic memory protection at the cost of external fragmentation; modern systems use paging exclusively with optional segment-like regions enforced by the OS.
- PIPT, VIVT, and VIPT represent cache addressing trade-offs between speed and correctness. VIPT with correct size constraints (size/associativity ≤ page size) is the dominant L1 cache design.
- SIMD — from Intel SSE/AVX to ARM SVE and NVIDIA SIMT — is the most pervasive form of parallelism in deployed hardware, accelerating everything from video codecs to neural networks.

Storage Systems:

- HDD, SATA SSD, and NVMe SSD represent a spectrum: capacity, balance, and performance respectively. NVMe over PCIe has become the standard for performance-critical storage.
- RAID 0 maximises speed with no redundancy; RAID 1 mirrors for reliability; RAID 5 balances capacity and single-disk tolerance; RAID 6 tolerates double failures; RAID 10 combines performance and reliability.
- RAID is not a substitute for backups — it protects only against disk hardware failure, not data corruption, accidental deletion, or site-level disasters.

Glossary

Term	Definition
Amdahl's Law	A law that states the maximum parallel speedup is 1/s, where s is the sequential fraction of a program that cannot be parallelised.
Superscalar Processor	A processor that issues multiple instructions per clock cycle using multiple execution units, with hardware dynamically finding independent instructions.
VLIW	Very Long Instruction Word — an architecture where the compiler explicitly packs multiple independent operations into a single long instruction for parallel execution.
Vector Processor	A processor with special vector registers and instructions that operate on entire arrays of data elements simultaneously.
SIMD	Single Instruction, Multiple Data — one instruction applies the same operation to multiple data elements in parallel using wide vector registers.
VIPT Cache	Virtually Indexed, Physically Tagged cache — uses virtual address for indexing (fast) and physical tag for comparison (correct), allowing parallel TLB and cache lookup.
RAID 5	A RAID level that stripes data across n disks with distributed XOR parity, providing (n−1)/n capacity utilisation and tolerance of any single disk failure.
Dragonfly Topology	A network topology with groups of all-to-all connected routers, linked sparsely between groups. Achieves very low diameter (3 hops) with moderate cabling cost.

ASID	Address Space Identifier — a tag added to TLB entries to identify the owning process, allowing TLB entries from different processes to coexist without a full flush on context switch.
NVMe	Non-Volatile Memory Express — a high-performance SSD interface protocol over PCIe that bypasses SATA/AHCI overhead, achieving sequential read speeds of 3,500–14,000 MB/s.

Self Assessment Questions

Section A: Multiple Choice Questions (MCQ)

1. What is the CPU execution time?

A) 0.5 seconds

B) 1 second

C) 2 seconds

D) 4 seconds

2. A program has a 5% sequential fraction. According to Amdahl's Law, what is the maximum speedup achievable?

A) 5×

B) 10×

C) 20×

D) 50×

3. Gustafson's Law differs from Amdahl's Law primarily because it assumes:

A) The number of processors is fixed

B) The problem size remains constant as processors increase

C) The problem size scales with the number of processors

D) The sequential fraction increases with more processors

4. Which interconnection network topology achieves very low diameter (typically 3 hops) and is used in the world's largest supercomputers?

A) 2D Mesh

B) Fat Tree

C) Dragonfly

D) Hypercube

5. Modern x86 processors (Intel Core, AMD Ryzen) handle the CISC instruction complexity by:

A) Executing CISC instructions directly with a complex ALU

B) Translating CISC instructions into RISC-like micro-operations internally

C) Using a separate RISC coprocessor for execution

D) Storing CISC instructions in cache before execution

6. Tomasulo's Algorithm is the theoretical foundation for which processor feature?

A) Branch prediction

B) Virtual memory management

C) Out-of-order execution and dynamic instruction scheduling

D) Vector processing

7. In VLIW processors, who is responsible for identifying and scheduling independent operations for parallel execution?

A) The hardware scheduler

B) The operating system

C) The compiler

D) The cache controller

8. Which cache addressing scheme allows TLB translation and cache indexing to proceed in parallel?

A) PIPT

B) VIVT

C) VIPT

D) CIPT

9. In a VIPT cache, to be alias-free the constraint must hold: cache size / associativity must be:

A) Greater than the page size

B) Equal to the page size

C) Less than or equal to the page size

D) Equal to the TLB size

10. What does ASID (Address Space Identifier) prevent in TLB management?

A) TLB misses on first access

B) The need for full TLB flushes on context switches

C) Page faults during program execution

D) Cache coherence problems in multiprocessors

11. An inverted page table stores one entry per:

A) Virtual page

B) Process

C) Physical frame

D) Segment

12. Intel AVX-512 operates on register widths of:

A) 128 bits

B) 256 bits

C) 512 bits

D) 64 bits

13. NVIDIA GPUs use SIMT (Single Instruction, Multiple Threads) where a group of 32 threads executing together is called a:

A) Block

B) Grid

C) Warp

D) Kernel

14. Which RAID level uses distributed parity and can survive failure of any single disk?

A) RAID 0

B) RAID 1

C) RAID 5

D) RAID 10

15. RAID 6 differs from RAID 5 by using:

A) Mirroring instead of parity

B) Striping without parity

C) Two independent parity calculations (P and Q)

D) Dedicated parity disks

16. A RAID 5 array with 6 disks of 4 TB each has a usable capacity of:

A) 24 TB

B) 20 TB

C) 12 TB

D) 16 TB

17. Which storage interface provides the highest bandwidth by connecting directly to CPU via PCIe lanes?

A) SATA III

B) USB 3.2

C) NVMe

D) SAS

18. A superscalar processor uses register renaming primarily to eliminate which type of hazard?

A) Compulsory miss hazards

B) WAR (Write After Read) and WAW (Write After Write) false dependencies

C) RAW (Read After Write) true dependencies

D) Structural hazards

19. The fat tree topology provides high bisection bandwidth because:

A) All nodes are directly connected to each other

B) Links become progressively wider (higher bandwidth) toward the root switches

C) It uses a fixed diameter of exactly 2 hops

D) It requires only one switch for all connections

20. RAID is NOT a substitute for backups because it does not protect against:

A) A single disk drive failure

B) Accidental file deletion and ransomware attacks

C) Read errors on a degraded array

D) Controller-detected bad sectors

Section B: True / False

State whether the following statements are TRUE or FALSE.

1. Gustafson's Law generally predicts higher speedup than Amdahl's Law for the same parallel fraction, because it assumes problem size scales with processor count.

2. RAID 0 provides better fault tolerance than a single disk because it distributes data across multiple drives.

3. In a superscalar processor, instructions are always completed in program order even though they may be executed out of order.

4. A VIPT cache with size/associativity equal to or less than the page size is completely alias-free and does not require TLB flushes on context switch.

5. VLIW processors achieve better binary compatibility across processor generations than superscalar processors.

Section C: Short Answer Questions

Answer each question in 3–5 sentences.

1. A program takes 100 seconds on a single processor. 80% of it can be parallelised. (a) Using Amdahl's Law, calculate the speedup with 4 and 16 processors. (b) What is the maximum possible speedup?

2. Explain the differences between superscalar and VLIW processors. Under what application scenarios would you choose VLIW over superscalar?

3. Compare PIPT, VIVT, and VIPT cache addressing schemes. Which is used for modern L1 data caches and why?

4. A storage system uses a 6-disk RAID 6 configuration with 8 TB drives. Calculate: (a) usable capacity, (b) number of disk failures it can tolerate, and (c) whether it is suitable for an archival system requiring high reliability.

5. What is the difference between Amdahl's Law and Gustafson's Law? Which is more appropriate for evaluating a weather simulation that scales its grid resolution with available processors?

Self Assessment: Answer Keys

MCQ Answer Key

Q.No	Correct Answer & Explanation
1	B) 1 second — CPU Time = IC × CPI × T_clk = $2\times10^9 \times 2 \times (1/4\times10^9) = 2\times10^9 \times 2 \times 0.25$ns = 1 second.
2	C) 20× — Maximum Speedup = 1/s = 1/0.05 = 20×. Even with infinite processors, the 5% sequential code caps speedup at 20×.
3	C) The problem size scales with the number of processors — Gustafson assumes larger workloads for more processors, producing optimistic and often more realistic speedup estimates for HPC.
4	C) Dragonfly — achieves diameter of 3 hops with all-to-all within groups and sparse inter-group connections. Used in Cray XC series and Fugaku.
5	B) Translating CISC instructions into RISC-like micro-operations — the front-end decodes x86 into μops; the back-end execution engine is RISC-style OOO superscalar.
6	C) Out-of-order execution and dynamic instruction scheduling — Tomasulo's algorithm uses reservation stations and the Common Data Bus to issue instructions as operands become available.
7	C) The compiler — VLIW relies entirely on compile-time static scheduling to fill instruction slots with independent operations.
8	C) VIPT — Virtually Indexed, Physically Tagged allows parallel cache indexing (virtual) and TLB lookup (physical tag), combining speed with correctness.
9	C) Less than or equal to the page size — this constraint ensures the index bits do not cross the virtual page boundary, preventing aliasing.

10	B) The need for full TLB flushes on context switches — ASID tags differentiate entries by process, so old process entries do not incorrectly match new process accesses.
11	C) Physical frame — inverted page table has one entry per physical memory frame (not per virtual page), making it proportional to physical RAM, not virtual address space.
12	C) 512 bits — AVX-512 uses 512-bit ZMM registers, accommodating 16 × 32-bit floats or 8 × 64-bit doubles simultaneously.
13	C) Warp — 32 threads executing the same instruction on different data simultaneously in SIMT execution.
14	C) RAID 5 — distributes XOR parity across all disks; any single disk failure can be reconstructed from remaining data and parity.
15	C) Two independent parity calculations (P and Q) — RAID 6 uses P (XOR) and Q (Reed-Solomon or Galois Field), tolerating any two simultaneous disk failures.
16	B) 20 TB — usable capacity = (n−1)/n × total = (6−1)/6 × 24 TB = 5/6 × 24 = 20 TB.
17	C) NVMe — NVMe SSD bypasses SATA/AHCI controller overhead and uses PCIe lanes directly, achieving 3,500–14,000 MB/s vs 550 MB/s for SATA.
18	B) WAR and WAW false dependencies — register renaming assigns new physical registers to destinations, eliminating false name-based dependencies while RAW true dependencies still require forwarding or stalls.
19	B) Links become progressively wider toward the root switches — this is the defining feature of fat tree, ensuring that aggregated traffic to/from any subtree has sufficient bandwidth at every level.

20	B) Accidental file deletion and ransomware attacks — RAID mirrors hardware failures only; it cannot protect against software-level data corruption, logical errors, or site disasters.

True / False Answer Key

Q.No	Answer / Explanation
1	TRUE — Gustafson's Law yields Scaled Speedup = p − s(p−1). For p = 100, s = 0.05: Gustafson = 100 − 0.05×99 ≈ 95×. Amdahl for same: 1/(0.05 + 0.95/100) ≈ 17×. Gustafson is far more optimistic because it scales the workload.
2	FALSE — RAID 0 provides NO fault tolerance. It actually has worse reliability than a single disk because the failure of ANY one disk in the stripe causes total data loss. MTTF = MTTF_single / n.
3	TRUE — Out-of-order processors use a Reorder Buffer (ROB) to track all in-flight instructions and commit (retire) results to the register file and memory in strict program order, even though internal execution was reordered.
4	TRUE — When size/associativity ≤ page size, the index bits used for cache lookup are contained entirely within the page offset bits of the virtual address, which are identical in virtual and physical addresses. No aliasing can occur.
5	FALSE — VLIW processors have poor binary compatibility. Changing the number of functional units or their latencies requires recompilation. A binary compiled for a 4-slot VLIW processor will not correctly utilise a 6-slot version. Superscalar processors maintain binary compatibility because the hardware handles scheduling dynamically.

Short Answer Key

Q.No	Model Answer
1	(a) s = 0.20 (20% sequential), parallel = 80%. With 4 processors: Speedup = 1/(0.20 + 0.80/4) = 1/(0.20+0.20) = 1/0.40 = 2.5×. Time = 100/2.5 = 40 sec. With 16 processors: Speedup = 1/(0.20 + 0.80/16) = 1/(0.20+0.05) = 1/0.25 = 4×. Time = 25 sec. (b) Maximum speedup (p→∞) = 1/s = 1/0.20 = 5×. Even infinite processors reduce time only to 20 seconds.
2	Superscalar: hardware dynamically detects and issues multiple independent instructions per cycle using out-of-order execution, ROB, and register renaming. Complex hardware, excellent binary compatibility. VLIW: compiler statically packs independent operations into fixed-slot instruction words; hardware is simple and power-efficient. Choose VLIW for DSP/signal processing applications (e.g., TI C6000 DSPs for telecom base stations) where: (1) access patterns are regular and statically predictable by the compiler, (2) hardware cost and power are constrained, and (3) binary compatibility across hardware generations is not required.
3	PIPT: both index and tag use physical address — correct but slow (requires TLB before cache access). VIVT: both virtual — fast but causes aliasing (full flush on context switch). VIPT: virtual index + physical tag — allows parallel TLB lookup and cache indexing; physical tag comparison ensures correctness. VIPT is standard for modern L1 data caches (e.g., ARM Cortex-A78, Intel Core) because it achieves PIPT-level correctness with near-VIVT speed, provided size/associativity ≤ page size.
4	(a) Usable capacity = (n−2)/n × total = (6−2)/6 × (6×8 TB) = 4/6 × 48 TB = 32 TB. (b) RAID 6 tolerates any simultaneous failure of exactly 2 disks. (c) Yes — RAID 6 is highly suitable for archival systems: it provides double-disk fault tolerance (critical given that modern 8 TB drives have 8–12 hour rebuild

	times during which a second failure would be catastrophic under RAID 5), delivers 67% capacity efficiency, and supports hot-spare drives for automatic rebuilding. For an archival system with long rebuild times, RAID 6 is the recommended choice.
5	Amdahl's Law assumes a fixed problem size: Speedup = 1/(s + (1−s)/p), giving max speedup of 1/s (pessimistic). Gustafson's Law assumes the problem scales with processors: Scaled Speedup = p − s(p−1), growing linearly. For weather simulation, the model grid resolution scales with available hardware — a 1000-node cluster runs a 1000× finer grid than a single node, not the same grid 1000× faster. Gustafson's Law is more appropriate here because the workload grows proportionally. The sequential fraction (I/O, model initialisation) remains nearly constant while the parallelisable compute scales, giving near-linear practical speedup — consistent with observed behaviour on systems like PARAM Rudra.

JGI JAIN DEEMED-TO-BE UNIVERSITY | ONLINE

ONLINE DEGREE PROGRAMS OFFERED

B.Com | BBA | MBA | MCA | M.Com | MA